LabClear 4.0.0-rc3 รายงานสถาปัตยกรรม เทคโนโลยี และ Agent Flow

วิชา 06048308 Intelligent Chatbot Development · Final Project

ผู้พัฒนา นายวัชรินทร์ บัวสอน (68076055) · นายศิริพล ศรีเฮงไพบูลย์ (68076060)

รุ่น 4.0.0-rc3 · branch `main` · source/test-evidence checkpoint `73d24b9cc4281b74829b63603786427d14cbe5c7` + dirty worktree · **วันที่** 11 ตุลาคม 2569

> รายงานนี้อธิบายระบบตามโค้ดจริงของรุ่น 4.0.0-rc3 ทุกตัวเลขอ่านจากไฟล์ใน repository (ระบุไฟล์ไว้ในวงเล็บ) ไม่ใช่ค่าที่ออกแบบไว้ล่วงหน้า ส่วนที่ปิดอยู่หรือยังไม่ได้ทำระบุไว้ตรง ๆ regression/UAT ใช้ข้อมูลสังเคราะห์และ mock/offline; live smoke แยกไว้ใน §17.2 อย่างละหนึ่งคำขอเพื่อยืนยัน connectivity/format เท่านั้น candidate นี้ยังไม่ได้ deploy

---

1. สรุป

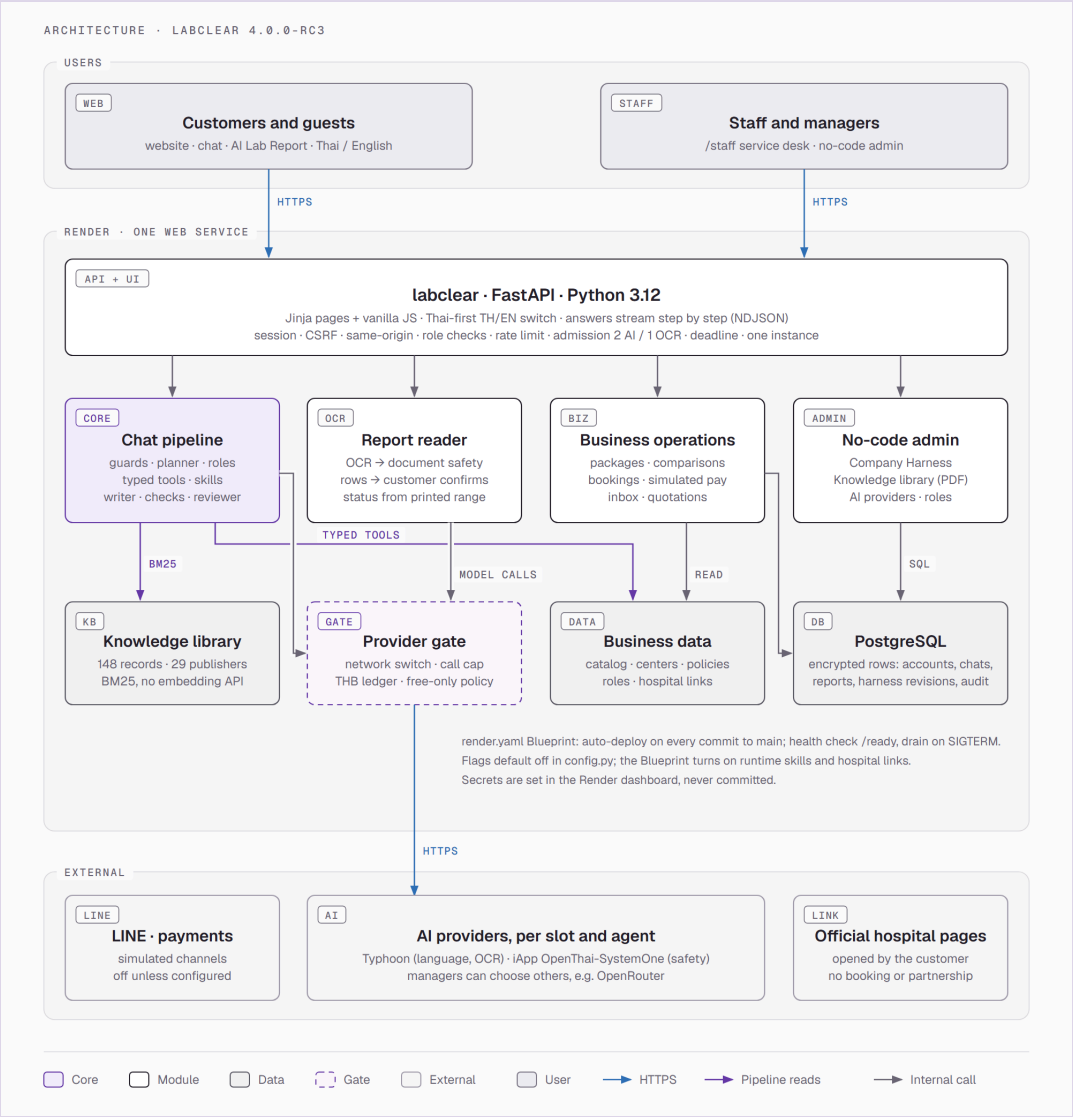
LabClear เป็นเว็บแอป FastAPI **บริการเดียว** บน Render ที่ส่งทั้งหน้าเว็บและ API แชทบอทตอบลูกค้าคลินิกตรวจสุขภาพจากข้อมูลของร้านและคลังความรู้ที่เจ้าของอนุมัติ อ่านใบผลแล็บ และอธิบายค่าเทียบช่วงอ้างอิงที่พิมพ์บนใบเดียวกัน หลักการออกแบบมี 6 ข้อ

หลักการ	ความหมายในโค้ด
โมเดลเสนอ Python ตัดสิน	planner เสนอ action บทบาท และคำค้น แต่โค้ดเลือกบทบาท ตรวจสิทธิ์ เรียกเครื่องมือ และสร้างตัวอย่างการจองเอง (`services/agent_tools.py`, `business_agent.py`)
ตอบจากหลักฐานเท่านั้น	ผู้เขียนอ้างได้เฉพาะรหัสหลักฐานที่ได้รับ ค่าและราคาถูกตรวจกับข้อมูลต้นทาง แล้ว reviewer ตรวจอีกชั้น
Fail closed	ไม่มีผลตัดสิน ผู้ให้บริการขัดข้อง หรือคำตอบผิดรูปแบบ ระบบหยุดและแจ้งเหตุ ไม่แต่งคำตอบแทน

หลักการ	ความหมายในโค้ด
ค่าใช้จ่ายมีเพดาน	ทุกการเรียกโมเดลผ่านด่านเดียว: สวิตช์เครือข่าย เพดานจำนวนครั้ง บัญชีบาท และนโยบายเรียกเฉพาะบริการฟรี
คำขอมีขอบเขตเวลา	ทุกคำขอ AI มี deadline เดียวทั้งเส้นทาง heartbeat และจบด้วยเหตุการณ์สุดท้ายหนึ่งรายการเสมอ
ภาษาไทยก่อน	หน้าเว็บ ข้อความผิดพลาด และคำตอบเป็นภาษาไทยเป็นค่าเริ่มต้น สลับภาษาอังกฤษได้

2. ภาพรวมระบบ

ผู้ใช้มี 2 กลุ่ม คือ ลูกค้า (รวมผู้เยี่ยมชมที่ไม่ได้เข้าสู่ระบบ) และเจ้าหน้าที่หรือผู้จัดการ ทั้งสองกลุ่มเข้าถึงบริการ `labclear` บน Render ผ่าน HTTPS บริการนี้รัน Uvicorn หนึ่ง process เพราะแชตของผู้เยี่ยมชมเก็บในหน่วยความจำของ process ข้อมูลถาวรอยู่ใน PostgreSQL ส่วนภายนอกมี 3 กลุ่ม คือ ผู้ให้บริการ AI (Typhoon, iApp) ช่องทางจำลอง (LINE, การชำระเงิน) และหน้าเว็บทางการของโรงพยาบาลที่ลูกค้าเปิดเอง



ภาพที่ 1 สถาปัตยกรรมของ LabClear รุ่น 4.0.0-rc3 (สร้างจากโค้ดด้วย `scripts/build_diagrams.py`)

ภายในบริการแบ่งเป็น 4 โมดูลหลัก

โมดูล	หน้าที่	ไฟล์หลัก
ไปป์ไลน์แชต	safety, planner, บทบาท, typed tools, runtime skills, ผู้เขียน, การตรวจ, reviewer	`services/business_agent.py`
ตัวอ่านใบผล	แปลงไฟล์ใน process แยก, OCR, ตรวจเอกสาร, แปลงเป็นแถว, ให้ลูกค้ายืนยัน	`services/report_reader_v2.py`, `document_worker.py`, `document_render.py`

โมดูล	หน้าที่	ไฟล์หลัก
งานธุรกิจ	แคตตาล็อก การเปรียบเทียบ การจอง การชำระเงินจำลอง กล่องข้อความ ใบเสนอราคา	`services/business_ops.py`, `business_store.py`, `business_plans.py`
หน้าผู้ดูแลแบบไม่ต้องเขียนโค้ด	Company Harness, คลังความรู้ (PDF), ผู้ให้บริการ AI, บทบาทผู้ช่วย	`routers/harness_admin.py`, `knowledge_admin.py`, `ai_admin.py`

3. Tech stack

3.1 Runtime และไลบรารี

Python 3.12 (`.python-version`; Render ตั้ง `PYTHON_VERSION=3.12.10`) ไม่มี Dockerfile หรือ pyproject ทุกไลบรารีของ runtime ปักเวอร์ชันใน `requirements.txt`

ไลบรารี	เวอร์ชัน	ใช้ทำอะไร
FastAPI	0.142.2	ASGI framework และ router (Starlette 1.7.0, Pydantic 2.14.0)
Uvicorn (standard)	0.54.0	ASGI server หนึ่ง process (`scripts/run_business.py`)
httpx	0.28.1	เรียกผู้ให้บริการ AI, Google OAuth token, Stripe sandbox
Jinja2	3.1.6	หน้าเว็บฝั่งเซิร์ฟเวอร์ (`templates/`)
pydantic-settings	2.15.0	ค่าตั้งทั้งหมดใน `config.py` พร้อมการตรวจขอบเขต
python-dotenv	1.2.4	อ่าน `.env` เฉพาะตอนรันเป็นบริการ
python-multipart	0.0.32	รับไฟล์อัปโหลด
Pillow	12.3.0	ตรวจและเข้ารหัสภาพใหม่ (ลบ metadata)
pypdfium2	5.14.0	แปลงหน้า PDF เป็นภาพ และภาพตัวอย่างของคลังความรู้
cryptography	50.0.2	Fernet เข้ารหัสข้อมูลที่จัดเก็บ
psycopg (binary)	3.2.10	ไดรเวอร์ PostgreSQL

ไลบรารี	เวอร์ชัน	ใช้ทำอะไร
reportlab	4.4.10	สร้าง PDF สรุปความรู้ในหน้าคลังความรู้

3.2 หน้าเว็บ

หน้าเว็บเป็น Jinja templates กับ JavaScript ธรรมดา ไม่มี framework ฝั่ง client และไม่มี build step ของแอป ไลบรารีภายนอกทุกตัวเก็บในเครื่อง (`static/vendor/`): marked 15.0.12 แปลง Markdown, DOMPurify 3.4.16 กรอง HTML และ three.js 0.186.1 สำหรับภาพ 3 มิติหน้าแรก ฟอนต์ทั้งหมด host เอง: IBM Plex Sans Thai (เนื้อหา), Trirong และ Source Serif 4 (หัวเรื่อง), Geist และ Geist Mono ข้อความภาษาไทย 3,187 รายการสร้างจาก `i18n/*.json` ด้วย `scripts/build_i18n.mjs`

ไฟล์ JavaScript	หน้าที่
`workspace.js`	พื้นที่ลูกค้า `/app` และศูนย์บริการลูกค้า `/staff`
`stream.js`	ตัวอ่าน NDJSON ที่จัดประเภทผลลัพธ์ (สำเร็จ, หน้า proxy, JSON error, สตรีมขาด) และ watchdog
`turns.js`	วาดข้อความ ขั้นตอน และความล้มเหลวของแต่ละรอบสนทนา
`api.js`	client กลางที่ส่ง CSRF และ guest token
`dock.js`	ผู้ช่วยแบบ dock บนทุกหน้าเว็บสาธารณะ
`admin-harness.js`	หน้า Company Harness และคลังความรู้ของผู้จัดการ
`i18n.js`, `theme.js`, `site.js`, `motion.js`	ภาษา ธีมสว่าง/มืด การค้น การเปรียบเทียบ และการเคลื่อนไหว

3.3 เครื่องมือทดสอบ

pytest 9.1.1 (`requirements-dev.txt`), Playwright 1.56.1 สำหรับชุดเบราว์เซอร์ (`package.json`; แอปไม่ใช้ Node), nbformat และ ipykernel สำหรับ notebook สาธิต (`requirements-eval.txt`) และ Tesseract สำหรับ OCR ตัวแทนในชุดทดสอบ OFFLINE เท่านั้น

4. การ deploy และ process

4.1 Render Blueprint

`render.yaml` มีบริการเดียวชนิด `web` ชื่อ `labclear` runtime Python แผน `free` ติดตาม branch `main` และ deploy อัตโนมัติทุก commit (`autoDeployTrigger: commit`) build ด้วย `pip install -r requirements.txt` เริ่มด้วย `python scripts/run_business.py` และตรวจสุขภาพที่ `/ready`

ค่าตั้งใน render.yaml	ค่า	ความหมาย
`APP_ENV`	production	เปิดการตรวจของ production เช่น ต้องมี PostgreSQL
`BUSINESS_EXTERNAL_ENABLED`	false	LINE และ Stripe จริงปิด
`RUNTIME_SKILLS_ENABLED`	true	ใช้ runtime skills
`HOSPITAL_LINKS_ENABLED`	true	แสดงลิงก์แพ็กเกจของโรงพยาบาล
`CHAT_DEADLINE_SECONDS` / `REPORT_DEADLINE_SECONDS`	220 / 150	deadline ต่อคำขอ
`STREAM_HEARTBEAT_SECONDS`	10	ระยะ heartbeat
`AI_MAX_IN_FLIGHT` / `OCR_MAX_IN_FLIGHT`	2 / 1	งานพร้อมกันสูงสุด
`PROVIDER_TRANSPORT_RETRIES`	0	ไม่ลองเรียกผู้ให้บริการซ้ำ
`SHUTDOWN_DRAIN_SECONDS`	20	เวลารองานค้างก่อนปิด

ค่าตั้งใน render.yaml	ค่า	ความหมาย
16 ค่าแบบ `sync: false`	ตั้งใน Dashboard	`DATABASE_URL`, `BUSINESS_DATA_KEY`, `PROVIDER_NETWORK_ENABLED`, คีย์ของ LLM, guard และ vision, `PROVIDER_BUDGET_CYCLE_ID`, `CLOUD_CALL_LIMIT`, `PROJECT_BUDGET_PRIOR_SPEND_THB`, ค่าบัญชีทดลอง และ Google OAuth

`MEDICAL_HARNESS_ENABLED` และ flag ของเอกสารองค์กร (`ORG_*`) ไม่อยู่ใน Blueprint จึงปิดตามค่าเริ่มต้นใน `config.py`

4.2 วงจรชีวิตของ process

1. `scripts/run_business.py` สร้าง Uvicorn หนึ่ง worker ที่ `0.0.0.0:$PORT` และ `timeout_graceful_shutdown` = drain + 5 วินาที
2. lifespan ใน `main.py` สร้าง schema ของฐานข้อมูลใน thread (จำกัด 15 วินาที) แล้วตั้งสถานะ started
3. `/health` คือ liveness ส่งรุ่นและ commit ส่วน `/ready` คือ readiness: ต้อง started ไม่อยู่ระหว่าง drain และ `SELECT 1` สำเร็จภายใน 1 วินาที (ผลที่สำเร็จเก็บ 2 วินาที) มิฉะนั้นตอบ 503
4. เมื่อได้ SIGTERM ครั้งแรก `DrainingServer` เริ่ม drain: `/ready` ตอบ 503 งานใหม่ได้ `server_draining` งานที่กำลังทำรอได้ 20 วินาทีแล้วถูกยกเลิก จากนั้น process จบด้วยสถานะ 0 สัญญาณครั้งที่สองหยุดทันที

5. โครงสร้างโค้ด

5.1 Routers

ไฟล์	Prefix	สิ่งที่ให้บริการ
`routers/business.py`	`/api/business`	session, บัญชี, แชต, ใบผล, การจอง, แผน, staff inbox, การชำระเงินและ webhook (56 endpoint)
`routers/business_ops.py`	`/api/business`	ค้นและเทียบแพ็กเกจ, การแจ้งเตือน, ปฏิทิน, ใบเสนอราคา PDF, dashboard, ลูกค้า, งบ, audit

ไฟล์	Prefix	สิ่งที่ให้บริการ
`routers/chats.py`	`/api/business`	รายการแชตและโปรเจกต์ (สูงสุด 30 โปรเจกต์)
`routers/ai_admin.py`	`/api/business/staff/ai-providers`	ผู้จัดการตั้งผู้ให้บริการต่อ slot และทดสอบ 1 ครั้ง
`routers/harness_admin.py`	`/api/business/staff/harness`	อ่าน บันทึก และย้อนรุ่น Company Harness
`routers/knowledge_admin.py`	`/api/business/staff/knowledge`	รายการความรู้ พัก/เปิด และหน้า PDF
`routers/public.py`	`/api/business/site`	ข้อมูลหน้าเว็บสาธารณะ ไม่มีข้อมูลส่วนบุคคล
`routers/site.py`	ไม่มี	หน้า HTML: `/`, `/packages`, `/compare`, `/centers`, `/lab-reports`, `/sources`, `/hospital-links` และอื่น ๆ
`routers/google_auth.py`	`/api/business/auth/google`	Google sign-in (PKCE S256, state cookie) สำหรับลูกค้าเท่านั้น
`routers/organization_sources.py`	`/api/business/organization-documents`	เอกสารขององค์กร (ตอบ 404 เมื่อปิด flag)
`routers/samples.py`	`/api/samples`	ภาพใบผลสังเคราะห์ 6 ใบ และการตรวจสิทธิ์เรียก AI

5.2 Services

กลุ่ม	ไฟล์
คำขอและความทนทาน	`execution.py` (request ID, deadline, admission, การยกเลิก, NDJSON, log), `request_limits.py`
Agent	`business_agent.py` (ไปป์ไลน์), `agent_tools.py` (typed tools), `runtime_skills.py`, `harness_config.py`, `business_dots.py` (บทบาท), `conversation_agent.py` (schema, ซ่อม JSON, ตรวจคำตอบ), `answer_checks.py`, `model_harness.py`
ผู้ให้บริการและค่าใช้จ่าย	`conversation_transport.py`, `providers.py`, `cost_ledger.py`, `free_policy.py`, `model_registry.py`

กลุ่ม	ไฟล์
ความปลอดภัย	`conversation_guard.py`, `trusted_origins.py`, `document_render.py`, `document_worker.py`
ความรู้	`evidence_search.py`, `knowledge_admin.py`, `hospital_links.py`, `organization_sources.py`
ใบผล	`report_reader_v2.py`, `lab_fields_v2.py`, `business_plans.py`
ข้อมูลและธุรกิจ	`business_store.py`, `guest_memory.py`, `chat_sessions.py`, `business_ops.py`, `business_documents.py`, `business_integrations.py`, `business_worker.py`, `demo_accounts.py`

6. วงจรของคำขอ

6.1 ชั้นก่อนถึง route

ทุกคำขอผ่าน middleware `RequestBoundary` แบบ ASGI ล้วน ซึ่งทำ 4 อย่าง: สร้างเลขอ้างอิงคำขอใหม่ทุกครั้งและส่งกลับใน `X-Request-ID`, จำกัดขนาด body (10 MB สำหรับ `/reports/read` และ `/chat/report`, 4 MB สำหรับ POST/PUT/PATCH อื่นของ `/api/business`) ทั้งจาก Content-Length และระหว่างอ่าน, ใส่ security headers และเขียน log หนึ่งบรรทัดแบบ JSON ต่อคำขอ

route ของ AI ตรวจเพิ่มอีก 3 ชั้น: origin ต้องตรงกับ host (หรืออยู่ใน `TRUSTED_ORIGINS`), session cookie (`HttpOnly`, `SameSite=Strict`) และ CSRF header `X-Business-CSRF` และ rate limit 120 คำขอต่อนาทีต่อ IP ในหน่วยความจำ

6.2 Execution context

`execution.start()` สร้าง context หนึ่งชุดต่อคำขอ AI ซึ่งถือ request ID, turn ID, deadline แบบ monotonic ของทั้งเส้นทาง, ช่องทำงาน (admission slot) และตัวยกเลิก

- **Admission** ถ้างาน AI เต็ม 2 งาน (หรือ OCR 1 งาน) คำขอใหม่ได้ `503 server_busy` พร้อม `Retry-After: 5` ทันที ไม่มีคิวและไม่เรียกผู้ให้บริการ

- **Deadline** แชต 220 วินาที อ่านใบผล 150 วินาที ครอบคลุมการอ่านเขียนฐานข้อมูล ทุก agent การเขียนใหม่ และการซ่อม JSON ขั้นใหม่จะไม่เริ่มถ้าเหลือเวลาน้อยกว่า 1 วินาที
- **การยกเลิก** ปุ่มหยุด การปิดการเชื่อมต่อ deadline และการปิดระบบยกเลิกการเรียกผู้ให้บริการที่ค้าง ฆ่า process อ่านไฟล์ และคืนช่องทำงาน
- **Storage นอก event loop** route ที่ใช้ฐานข้อมูลอย่างเดียวเป็นฟังก์ชันปกติที่ FastAPI รันใน thread ส่วนในไปป์ไลน์ใช้ `offload()` และ PostgreSQL ตั้ง `statement_timeout=15000` และ `lock_timeout=10000`

6.3 สัญญา NDJSON

เมื่อเบราว์เซอร์ส่ง `Accept: application/x-ndjson` คำตอบเป็นสตรีมบรรทัด JSON ที่มีลำดับแน่นอน

เหตุการณ์	เมื่อไร	ข้อมูล
`accepted`	บรรทัดแรกทันทีที่รับงาน	`request_id`, `deadline_ms`, `heartbeat_ms`
`step`	ทุกขั้นเริ่มและจบ	`id`, `state` (running, done, error), `label`, `detail`, `elapsed_ms`, `duration_ms`
`heartbeat`	ทุก 10 วินาทีที่ไม่มีเหตุการณ์อื่น	`elapsed_ms`
`done`	สำเร็จ (บรรทัดสุดท้าย)	`result`, `request_id`, จำนวนเหตุการณ์ที่ถูกตัดถ้ามี
`error`	ล้มเหลว (บรรทัดสุดท้าย)	`code`, `message`, `status`, `origin`, `request_id`, `step`

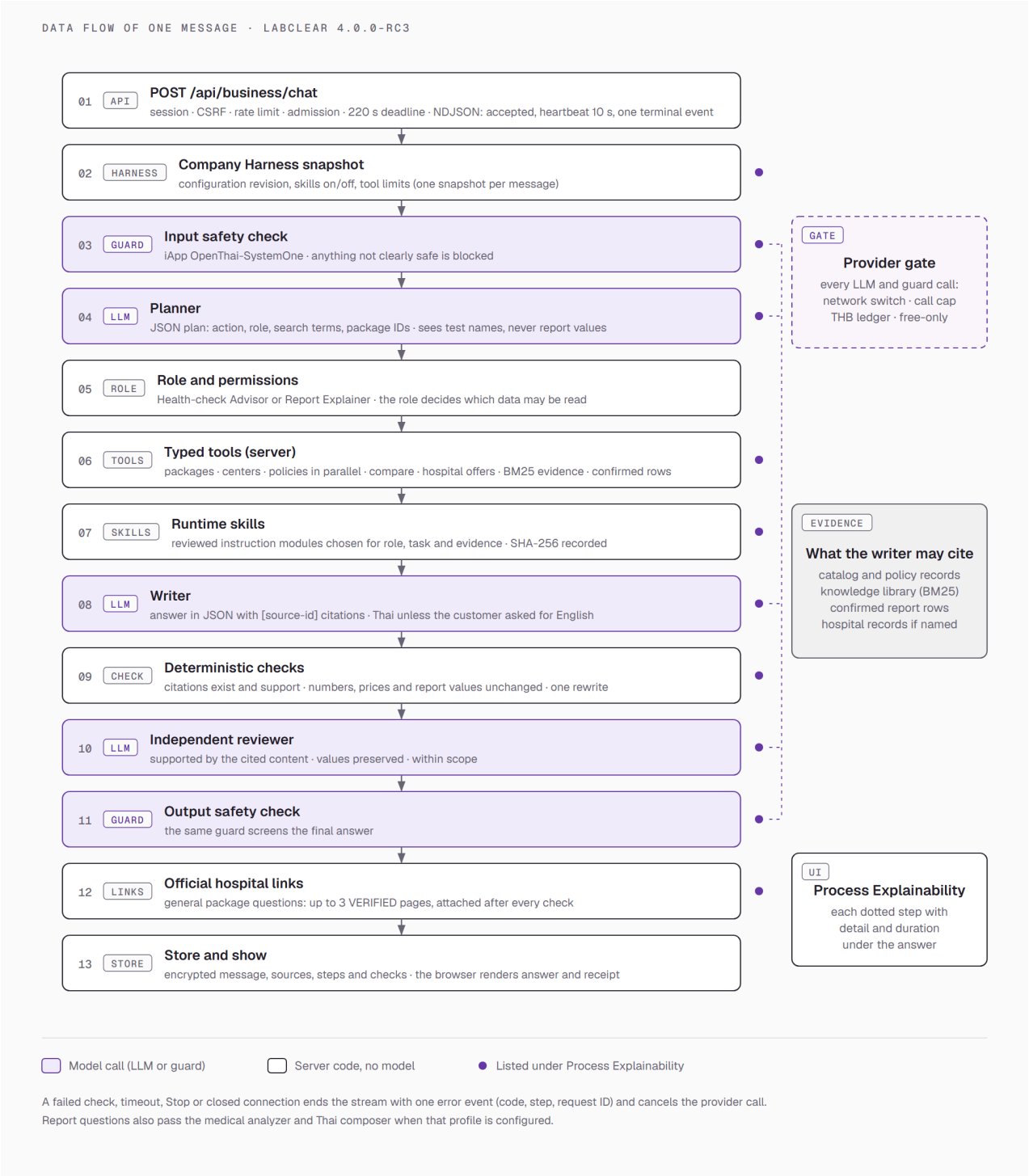
ทุกสตรีมจบด้วย `done` หรือ `error` เพียงหนึ่งบรรทัด คิวภายในจำกัด 256 เหตุการณ์ ฝั่งเบราว์เซอร์ `stream.js` มี watchdog 2 ตัว (เงียบเกิน 35 วินาที และ deadline + 10 วินาที) แยกหน้า HTML ของ proxy, JSON error, สตรีมที่ขาดก่อนบรรทัดสุดท้าย และบรรทัดที่อ่านไม่ได้ ข้อความของลูกค้ากลับไปที่ช่องพิมพ์เมื่อคำขอล้มเหลว

6.4 รหัสข้อผิดพลาด

ข้อผิดพลาดทุกตัวเป็น `ConversationError` ที่มี `code`, `status` และ `origin` (client, app หรือ upstream) ตัวอย่างกลุ่มหลัก

กลุ่ม	รหัส (HTTP)
ความทนทาน	`server_busy` 503, `server_draining` 503, `request_timeout` 504, `cancelled` 409/499, `storage_unavailable` 503
ผู้ให้บริการ	`upstream_timeout` 504, `upstream_unavailable` 502, `upstream_rate_limited` 503, `provider_rejected` 502, `provider_response_invalid` 502, `offline` 503
งบและนโยบาย	`budget_exhausted` 429, `cycle_required` 503, `free_policy_blocked` 409, `free_quota_exhausted` 429
การตรวจคำตอบ	`citation_invalid`, `observation_invalid`, `price_invalid`, `evidence_review_failed`, `evidence_missing`, `review_failed`, `role_violation` (502) และ `safety_blocked` 422
ใบผล	`file_too_large` 413, `unsupported_image` 415, `page_limit` 422, `document_timeout` 422, `not_a_report` 422

7. Agent flow ของหนึ่งข้อความ



ภาพที่ 2 การไหลของข้อมูลของหนึ่งข้อความ (กล่องสีม่วงเรียกโมเดล จุดสีม่วงแสดงใน Process Explainability)

7.1 Agent และ slot ของผู้ให้บริการ

ระบบมี 3 slot หลักและ agent ที่ตั้งผู้ให้บริการแยกได้ agent ที่ยังไม่ได้ตั้งใช้ค่าของ slot `llm` ผู้จัดการตั้งค่าได้ที่หน้าผู้ให้บริการ AI คีย์ถูกเข้ารหัสและ

แสดงเพียง 4 ตัวท้าย

Agent / slot	หน้าที่	ผลลัพธ์	ค่าเริ่มต้น
`guard`	ตรวจข้อความเข้า คำตอบ และข้อความจากเอกสาร	ป้ายความปลอดภัย	iApp `openthai-systemone`
`agent_plan`	Planner: action, บทบาท, คำค้น, รหัสแพ็กเกจ	JSON `Plan` (≤900 tokens)	Typhoon `typhoon-v2.5-30b-a3b-instruct`
`agent_advisor`	Health-check Advisor เขียนคำตอบเรื่องบริการ	JSON `Answer`	เหมือน `llm`
`agent_explainer`	Report Explainer อธิบายใบผลที่ยืนยันแล้ว	JSON `Answer`	เหมือน `llm`
`agent_review`	Reviewer ตรวจหลักฐาน ค่า และขอบเขต	JSON `EvidenceReview` (≤300 tokens)	เหมือน `llm`
`vision`	อ่านภาพใบผล หน้าละครั้ง	ข้อความ	Typhoon `typhoon-ocr`
`agent_medical_analyzer`, `agent_thai_composer`	วิเคราะห์ใบผลแบบมีโครงสร้างแล้วเรียบเรียงไทย	JSON ที่ตรวจด้วย schema	ปิด (ต้องตั้งโมเดลและราคาที่ตรวจแล้ว)

ผู้ให้บริการที่เลือกได้มี 15 แบบ เช่น Typhoon, OpenAI, Anthropic, Gemini, OpenRouter, DeepSeek, Qwen และ Llama Guard (`services/providers.py`) ทุกคำขอใช้ snapshot ของการตั้งค่าชุดเดียวตลอดเส้นทาง

7.2 ลำดับขั้นของ `business_agent.run`

#	ขั้น (step id)	สิ่งที่เกิดขึ้น	เรียกโมเดล
1	`harness`	อ่าน snapshot ของ Company Harness (รุ่นและ SHA-256)	–
2	`safety_in`	regex ภาษาไทยและอังกฤษ ถ้าพบคำสั่งแทรกหยุดทันทีแล้วส่งข้อความให้ guard	guard
3	`plan`	planner เห็นข้อความ ประวัติ 12 รอบ และชื่อการตรวจในใบผล (ไม่เกิน 40 ชื่อ ไม่เห็นค่า)	planner
4	–	โค้ดตัดสินบทบาท: คำขออธิบายใบผลไปที่ Explainer, คำถามที่ระบุชื่อแพ็กเกจไปที่บทบาทที่อ่านแคตตาล็อก, action ที่บทบาททำไม่ได้ย้ายไปบทบาทที่ทำได้หรือกลายเป็นคำถามกลับ	–
5	`tool_*`, `data`	`lookup_packages`, `lookup_branches`, `lookup_policies` พร้อมกันตามสิทธิ์ของบทบาท, `compare_packages` เมื่อมี 2–4 แพ็กเกจ, `get_external_hospital_offer` เมื่อข้อความเอ่ยชื่อโรงพยาบาล	–
6	`search`	`retrieve_evidence` ค้น BM25 เมื่อมีคำค้นและบทบาทอ่านความรู้ทางการแพทย์ได้	–
7	`report`	`get_confirmed_report_rows` เมื่อมีใบผลที่ยืนยันแล้วและบทบาทอ่านใบผลได้	–
8	–	สร้างตัวอย่างการกระทำจากข้อมูลเซิร์ฟเวอร์ (`preview_booking` สำหรับจองหรือเสนอราคา) และปุ่มลัดไม่เกิน 2 ปุ่ม	–
9	`skills`	เลือก runtime skills ตามบทบาท งาน และหลักฐาน	–
10	`analyze`	(เมื่อเปิด medical harness) วิเคราะห์ใบผลแล้วให้ Thai composer เป็นผู้เขียน	analyzer
11	`draft`	ผู้เขียนตามบทบาทเขียน JSON พร้อม `[source-id]` ความยาวตาม Harness (ค่าเริ่มต้น 2,400 tokens)	ผู้เขียน

#	ขั้น (step id)	สิ่งที่เกิดขึ้น	เรียกโมเดล
12	–	ตรวจในโค้ด: citation ต้องอยู่ในหลักฐาน (การแพทย์ ≤8, รวม ≤30), ไม่มีลิงก์/HTML/ภาพ, ค่าตรงแถวที่ยืนยัน, จำนวนเงินรู้จัก, ข้อความทางการแพทย์ต้องมีแหล่งการแพทย์, บทบาทที่ห้ามขายไม่ขาย และเติมคำแนะนำค่าวิกฤต	–
13	`review`	reviewer ตรวจ supported, values_preserved, within_scope	reviewer
14	–	ถ้าขั้น 12–13 ไม่ผ่านในรอบแรก เขียนใหม่ได้ 1 ครั้ง แล้วตรวจทุกข้ออีกครั้ง (ไม่เขียนใหม่เมื่อ guard ปฏิเสธ)	ผู้เขียน, reviewer
15	`safety_out`	guard ตรวจคำตอบ คำถามต่อ ตัวอย่างการกระทำ และเหตุผลของแผน	guard
16	`offers`	คำถามแพ็กเกจทั่วไป: แนบลิงก์โรงพยาบาลสถานะ VERIFIED ที่มีราคาไม่เกิน 3 รายการ หลังผ่านทุกการตรวจ	–

route `turn()` ครอบไปป์ไลน์นี้: ตั้งสถานะ busy ของบทสนทนา (เวลาที่เหลือ + 5 วินาที), ตัดคำตอบที่มาช้าถ้าบทสนทนาเปลี่ยนรุ่นหรือ turn ID แล้ว, เก็บตัวอย่างการกระทำ 600 วินาที, บันทึกคำตอบพร้อมแหล่ง ขั้นตอน และผลการตรวจ หรือทำเครื่องหมายว่าลองใหม่ได้เมื่อรหัสอยู่ในกลุ่ม retryable

7.3 จำนวนการเรียกโมเดล

ข้อความปกติเรียก 5 ครั้ง (guard ขาเข้า, planner, ผู้เขียน, reviewer, guard ขาออก) กรณีมากที่สุด 12 ครั้ง เพราะการเรียกแบบ JSON แต่ละครั้งซ่อมรูปแบบได้ 1 ครั้ง และคำตอบเขียนใหม่ได้ 1 ครั้ง ถ้าเปิด medical analyzer เพิ่มอีก 1 ครั้ง ถ้า regex พบคำสั่งแทรก ระบบหยุดก่อนเรียกโมเดลใด ๆ

7.4 เส้นทางใบผลแล็บ

ใบผลใช้ 2 คำขอเพื่อให้ลูกค้ายืนยันค่าก่อนเสมอ

1. `POST /chat/report` รับ 1–3 ไฟล์หรือรายงานตัวอย่าง ตรวจขนาดและ magic bytes ขอช่อง OCR แล้วส่งไฟล์ให้ process แยกแปลงเป็นภาพ (ขั้น `prepare`) จองสิทธิ์อ่านตามแผน (คืนสิทธิ์ถ้าล้มเหลว)

2. `report_reader_v2.read_report`: Typhoon OCR หน้าละครั้ง (≤3 หน้า) → guard ตรวจข้อความในฐานะเอกสาร → โมเดลภาษาแปลงเป็นแถว → guard ตรวจแถวอีกครั้ง → Python ปรับรูปแบบและคำนวณสถานะจากช่วงที่พิมพ์ ระบบบันทึกเป็นร่างและแสดงการ์ดค่า ยังไม่อธิบาย
3. `POST /chat/report/confirm` รับค่าที่ลูกค้าแก้ (ถ้ามี) ตั้ง confirmed แล้วเรียก `turn()` โดยบังคับบทบาท Report Explainer

การอ่านใบผลด้วย Typhoon เรียกโมเดลได้สูงสุด 7 ครั้ง (OCR 3, guard 1, แปลงแถว 2, guard 1)

8. Typed tools

เครื่องมือทั้งหมดอยู่ใน `services/agent_tools.py` schema รุ่น `labclear-tools-1.1.0` อินพุตเข้มงวด (ห้ามฟิลด์เกิน รหัสแพ็กเกจต้องตรง `^P\d{2}$`) บทบาทเรียกได้เฉพาะเครื่องมือที่ scope อยู่ในสิทธิ์อ่านของบทบาท

Tool	Scope	Advisor	Explainer	เวลา (วินาที)	รายการสูงสุด	ขนาดสูงสุด (ตัวอักษร)
`lookup_packages`	catalog	✓	–	2	40	60,000
`compare_packages` (2–4 รหัส)	catalog	✓	–	2	1	20,000
`lookup_branches`	branches	✓	–	2	1	20,000
`lookup_policies`	policies	✓	✓	2	1	20,000
`retrieve_evidence` (คำค้น ≤700, 1–8 รายการ)	medical	✓	✓	10	8	60,000
`get_confirmed_report_rows`	report	–	✓	2	1	80,000
`preview_booking` (ต้องมีสิทธิ์ quote/book)	catalog	✓	–	2	1	20,000

Tool	Scope	Advisor	Explainer	เวลา (วินาที)	รายการสูงสุด	ขนาดสูงสุด (ตัวอักษร)
`get_external_hospital_offer` (เมื่อเปิด flag)	catalog	✓	–	2	20	20,000

ทุกการเรียกบันทึก audit: ชื่อ รุ่น scope บทบาท SHA-256 ของอาร์กิวเมนต์ 16 ตัวแรก ผล จำนวนรายการ ขนาด การตัดทอน รหัสผิดพลาด และเวลา โดยไม่เก็บอาร์กิวเมนต์จริง audit นี้แสดงใน Process Explainability ผู้จัดการลดเวลาและจำนวนรายการได้แต่เพิ่มเกินค่าที่ลงทะเบียนไม่ได้ (`min(registered, harness)`) และปิดเครื่องมือได้ (`tool_disabled`)

9. Runtime skills และ Company Harness

9.1 Runtime skills

ชุดคำสั่ง `labclear-thai-health-communication` รุ่น 0.3.0-offline (`runtime_skills/thai_health/manifest.json`) มี 8 โมดูล ทุกไฟล์มี SHA-256 ใน manifest และถูกตรวจทุกครั้งที่อ่าน ถ้าไม่ตรงได้ `skill_invalid` ลำดับโมดูลคงที่ และคำตอบบันทึกรุ่นของชุด รายชื่อโมดูล และ hash 16 ตัวแรก

โมดูล	รุ่น	เลือกเมื่อ
`core` (ล็อก)	0.2.0	ทุกข้อความ
`thai-style`	0.2.0	ทุกข้อความ (ทำตามเมื่อลูกค้าขอภาษาอื่น)
`evidence-citation` (ล็อก)	0.3.0	มีหลักฐานใดก็ตาม
`scope-uncertainty` (ล็อก)	0.3.0	มีหลักฐานการแพทย์ ใบผล หรือ action urgent/clarify
`lay-explanation`	0.3.0	มีหลักฐานการแพทย์และไม่มีใบผลที่ยืนยัน
`patient-explanation`	0.2.0	มีใบผลที่ยืนยันแล้ว และบทบาทอ่านใบผลได้
`package-advice`	0.2.0	มีข้อมูลแพ็กเกจ และบทบาทเสนอราคาได้
`package-compare`	0.3.0	`compare_packages` สำเร็จ และบทบาทเสนอราคาได้

สถานะการตรวจของชุดคำสั่ง: clinical validation `NOT_RUN` และ language review `PENDING`

9.2 Company Harness

ผู้จัดการปรับการทำงานของผู้ช่วยได้โดยไม่แก้โค้ด (`services/harness_config.py`)

การตั้งค่า	ขอบเขต	ค่าเริ่มต้น
ใช้ runtime skills	เปิด/ปิด	ตาม `RUNTIME_SKILLS_ENABLED`
จำนวนแหล่งอ้างอิงต่อการค้น	1–8	6
ความยาวคำตอบสูงสุด	500–4,000 tokens	2,400
แต่ละ skill	เปิด/ปิด และถ้อยคำบริษัทไม่เกิน 4,000 ตัวอักษร	เปิด
แต่ละ tool	เปิด/ปิด เวลา 0.1–10 วินาที รายการ 1–40 (ไม่เกินค่าที่ลงทะเบียน)	2 วินาที, 8 รายการ

สิ่งที่เปลี่ยนไม่ได้: ปิดโมดูลที่ล็อก เพิ่มขีดจำกัดเกินค่าในโค้ด เพิ่ม skill หรือ tool ที่ไม่รู้จัก และสิทธิ์ของบทบาท ถ้อยคำของบริษัทต่อท้ายคำสั่งในฐานะ "ต่ำกว่านโยบายของแอป" ทุกการบันทึกตรวจรุ่นแบบ optimistic (`revision_conflict` 409) เก็บรุ่นก่อนหน้า บันทึก audit และย้อนกลับได้ (แสดง 20 รุ่นล่าสุด) SHA-256 ของการตั้งค่าบันทึกไว้ในทุกคำตอบ

10. ด่านผู้ให้บริการและค่าใช้จ่าย

ทุกการเรียกผู้ให้บริการผ่าน `post_json` ใน `services/conversation_transport.py` ตามลำดับนี้

1. endpoint ต้องเป็น HTTPS และ URL ที่กำหนดเองต้องไม่ชี้ IP ภายใน
2. นโยบายเรียกเฉพาะบริการฟรี (เมื่อกำหนด `FREE_ONLY_POLICY_PATH`): host, path และโมเดลต้องตรงตัว สถานะ `VERIFIED_FREE_FOR_THIS_ACCOUNT` อายุไม่

เกิน 7 วัน และโควตาต่อนาทีและต่อ run ที่ทุกบทบาทใช้ร่วมกัน

3. `PROVIDER_NETWORK_ENABLED` (ปิดเป็นค่าเริ่มต้น ได้ `offline` 503) และเพดานจำนวนครั้ง `CLOUD_CALL_LIMIT` ต่อรอบงบ `PROVIDER_BUDGET_CYCLE_ID` (นับแม้การเรียกล้มเหลว เกินได้ `budget_exhausted` 429)
4. จองค่าใช้จ่ายในบัญชีบาท `PROJECT_BUDGET_THB` (300 บาท) จากค่าประมาณ (ขนาดข้อความ + 256 + ภาพละ 1,500 tokens และ max_tokens ขาออก) ต้องระบุค่าใช้จ่ายก่อนหน้า `PROJECT_BUDGET_PRIOR_SPEND_THB`
5. ส่ง HTTP หนึ่งครั้ง ไม่ตาม redirect

เรื่อง	ค่า
เวลาต่อการเรียก	min(timeout ของ slot, 75 วินาที, เวลาที่เหลือของคำขอ) ค่า slot: LLM 60, vision 60, guard 20 วินาที
การลองใหม่	ไม่มี (`PROVIDER_TRANSPORT_RETRIES=0`) และไม่มีผู้ให้บริการสำรอง มีเพียงการซ่อม JSON 1 ครั้งและการเขียนคำตอบใหม่ 1 ครั้ง
ขนาดคำตอบ	ไม่เกิน 1,000,000 bytes และข้อความ 50,000 ตัวอักษร ปฏิเสธ `finish_reason` แบบ length หรือ content_filter
การปิดบัญชี	succeeded, failed หรือ cancelled การเรียกที่ล้มเหลวหรือถูกยกเลิกคิดเต็มค่าประมาณ และปิดบัญชีซ้ำไม่ได้
OpenRouter สำหรับ agent ใหม่	ต้องระบุ provider allowlist ส่ง `allow_fallbacks=false`, `data_collection=deny` และ zero data retention

11. คลังความรู้และ RAG

คลังความรู้ `knowledge/evidence/catalog.json` (schema `resultscope-evidence-v2`, รุ่น `2026-10-09-owner-approved-148`) มี 148 รายการจาก 29 ผู้เผยแพร่ (58 รายการเดิมและ 90 รายการที่เจ้าของอนุมัติ) แบ่งเป็น public_reference 80 รายการและ public_education 68 รายการ

เรื่อง	รายละเอียด
วิธีค้น	BM25 (k1=1.5, b=0.75) บนชื่อเรื่อง aliases และเนื้อหา ไม่มี embedding หรือ vector database

เรื่อง	รายละเอียด
การตัดคำ	คำละติน `[a-z0-9]+` และคู่อักษรไทย (bigram) คำถามภาษาไทยจึงเจอรายการภาษาอังกฤษผ่าน aliases ภาษาไทย
จำนวนผลลัพธ์	ค่าเริ่มต้น 6 ปรับใน Harness ได้ 1–8
ความถูกต้องตอนโหลด	รหัสไม่ซ้ำ, ชนิดข้อมูลสาธารณะเท่านั้น, SHA-256 ของเนื้อหาและไฟล์ต้นฉบับตรง, URL เป็น https ถ้าไม่ผ่านได้ `evidence_unavailable`
การจัดการ	ผู้จัดการเปิดอ่านเป็น PDF (ตรวจ hash ซ้ำ) และพักรายการได้ รายการที่พักไม่ถูกค้น

ลิงก์โรงพยาบาล (`business_data/hospital_links.json`) เก็บข้อเสนอจากหน้าเว็บทางการ 7 รายการ สถานะคำนวณทุกครั้ง (VERIFIED, UNVERIFIED, EXPIRED_SALE, STALE และอื่น ๆ) รายการที่ไม่ใช่ VERIFIED ถูกลบราคา host ต้องอยู่ใน allowlist 6 โดเมน และทุกรายการระบุว่าไม่ใช่พันธมิตรและจองผ่าน LabClear ไม่ได้

12. ข้อมูลและการจัดเก็บ

เรื่อง	รายละเอียด
ฐานข้อมูล	PostgreSQL (psycopg 3, connect_timeout 10) เมื่อมี `DATABASE_URL` มิฉะนั้น SQLite `data/business.sqlite3` (busy timeout 10 วินาที) บน cloud ต้องมี PostgreSQL
ตาราง	`rs_entities(id, kind, owner, state, branch, payload, created)` เก็บทุก record และ `rs_mutex` สำหรับ lock ทั่วระบบ มี index `(kind, owner)`
ชนิดข้อมูล	บัญชีและ session, บทสนทนาและโปรเจกต์, นัด ใบผล ใบเสนอราคา การแจ้งเตือน audit, การชำระเงินจำลอง, LINE จำลอง, การตั้งค่า (AI, Harness, ความรู้), งบและโควตา
การเข้ารหัส	payload ทั้งก้อนเข้ารหัส Fernet ด้วย `BUSINESS_DATA_KEY` เหลือเพียง id, kind, owner, state, branch, created เป็นข้อความธรรมดา
รหัสผ่านและ session	PBKDF2-SHA256 310,000 รอบ เกลือ 16 bytes token ของ session เก็บเป็น SHA-256

เรื่อง	รายละเอียด
ผู้เยี่ยมชม	อยู่ในหน่วยความจำของ process เท่านั้น อายุ 20 นาทีนับจากการใช้ครั้งล่าสุด สูงสุด 100 คนและ 64 MiB ลบเมื่อรีเฟรช ปิดหน้า หรือเข้าสู่ระบบ token อยู่ในหน่วยความจำของหน้าเว็บ

13. ความปลอดภัย

ด้าน	มาตรการ
Session	cookie `labclear_session` HttpOnly, SameSite=Strict, Secure บน cloud, อายุ 1 วัน; CSRF ตรวจด้วย `hmac.compare_digest`
Origin	ปฏิเสธ `Sec-Fetch-Site: cross-site`; Origin ต้องตรงกับ Host หรืออยู่ใน `TRUSTED_ORIGINS` แบบตรงตัว
อัตราการใช้	120 คำขอต่อนาทีต่อ IP; เข้าสู่ระบบ 8 ครั้งต่อ 15 นาที
ไฟล์อัปโหลด	1–3 ไฟล์ ไฟล์ละ ≤3 MB; PDF, PNG, JPEG ตาม magic bytes; ≤3 หน้า; ≤12 ล้านพิกเซล; เข้ารหัสภาพใหม่และลบ metadata
Process อ่านไฟล์	`python -I services/document_render.py` env เหลือเพียงค่าพื้นฐาน ไม่มี secret; RLIMIT_AS 384 MB, RLIMIT_CPU ตามเวลา, RLIMIT_FSIZE 0; เวลาไม่เกิน 30 วินาทีหรือเวลาที่เหลือ; ถูกฆ่าและเก็บกวาดเมื่อหมดเวลาหรือยกเลิก
Guard	regex คำสั่งแทรก (ไทยและอังกฤษ) และ safety model ทั้งขาเข้า ขาออก และเอกสาร; fail closed
คำตอบ	ตัด HTML ลิงก์ ภาพ และ URL ก่อนแสดง; เบราว์เซอร์กรองด้วย DOMPurify
Headers	`X-Request-ID`, `nosniff`, `Referrer-Policy: no-referrer`, `Permissions-Policy`, `X-Robots-Tag: noindex`, CSP `default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'`
Webhook	ลายเซ็น HMAC พร้อมหน้าต่างเวลา 300 วินาที (Stripe sandbox และตัวจำลองการชำระเงิน)
Log	หนึ่งบรรทัด JSON ต่อคำขอด้วยรายการคีย์ที่อนุญาต ไม่มีข้อความลูกค้า ข้อมูลใบผล หรือคีย์

14. ความทนทาน

ค่าตั้งของความทนทานอยู่ใน `config.py` พร้อมขอบเขตที่ตรวจตอนเริ่ม

ค่า	ค่าเริ่มต้น	ขอบเขต
`CHAT_DEADLINE_SECONDS`	220	30–600
`REPORT_DEADLINE_SECONDS`	150	30–600
`STREAM_HEARTBEAT_SECONDS`	10	2–30
`AI_MAX_IN_FLIGHT`	2	1–16
`OCR_MAX_IN_FLIGHT`	1	1–16 และไม่เกิน AI
`PROVIDER_TRANSPORT_RETRIES`	0	0 เท่านั้น
`DOCUMENT_WORKER_SECONDS`	30	2–120
`DOCUMENT_WORKER_MEMORY_MB`	384	192–4,096
`SHUTDOWN_DRAIN_SECONDS`	20	1–25

ชุดทดสอบ R01–R12 (`scripts/benchmark_resilience.py --offline`) จำลองผู้ให้บริการค้าง ตอบช้า ตอบ 502/503/429 หรือ JSON เสีย, หน้า HTML ของ proxy, สตรีมขาด, ปุ่มหยุดและการปิดหน้า, ผู้ใช้เกินขีดจำกัด, PDF เสียหรือใหญ่เกิน, ฐานข้อมูลช้า, SIGTERM, การกดลองใหม่ซ้ำ และ cold start ผลบน commit `3191f93`: ผ่าน 12/12 (183 assertion) ไม่มีการเชื่อมต่อออก score hash `7ec88f1376d2d778` เท่ากันทั้งสอง seed ชุดในเบราว์เซอร์จริงผ่าน 10/10 คะแนนนี้คืออัตราผ่านของชุดทดสอบแบบกำหนดได้ ไม่ใช่ uptime และไม่ใช่เวลาบน Render

เวลาของระบบเองในเครื่องทดสอบ (ไม่รวมเวลาโมเดล ไม่ใช่ Render): `/ready` พร้อมใน 673 ms หลังเริ่ม process, แชต p50 73 ms และ p95 91 ms, สองงานพร้อมกัน p50 173 ms, อ่านใบผล p50 239 ms, หน่วยความจำสูงสุด 81 MiB, ปิดหลัง SIGTERM 366 ms

15. การทดสอบและเครื่องมือ

ชุด	ที่อยู่	ผลบนรุ่นนี้
Unit และ API	`tests/test_*.py` (27 ไฟล์) ผ่าน `scripts/offline_check.py` ที่ปิดการเชื่อมต่อออก	547 ผ่าน, 1 warning
Browser UAT	`tests/browser/uat.cjs`	GL-03 historical run retained for context; fresh `final03` is 37 PASS / 0 FAIL / 0 browser errors, exit code 0 in local mocked scope
Browser upgrade	`tests/browser/upgrade.cjs`	10/10
ไทย/อังกฤษ	`tests/browser/i18n_audit.mjs` (390, 768, 1440 px)	43/43
ความทนทาน	`tests/resilience/` และ `tests/browser/resilience.cjs`	12/12 และ 10/10
ชุดตามโจทย์	`scripts/benchmark_labclear.py` กับ `eval/coursework/` (OFFLINE, REPLAY, LIVE_FREE)	OFFLINE 15/20 ทุกโปรไฟล์; LIVE_FREE ถูกระงับที่ preflight
ชุดไฟล์ใบผล	`eval/ocr_files/` 12 ไฟล์	OCR ตัวแทนอ่านตรง 217/252 ค่า
Notebook	`notebooks/LabClear_Harness_Demo.ipynb`	แสดง prompt, tools, outputs และ rollout ของความทนทาน

สคริปต์สำคัญ: `run_business.py` (เริ่มบริการ), `dev_mock_api.py` (แอปจริงกับตัวแทนโมเดล), `boot_check.py` (ตรวจ entry point ของ Render), `score_benchmark.py` (ให้คะแนนแบบกำหนดได้), `measure_resilience.py` (วัดเวลาของระบบเอง), `build_diagrams.py`, `build_report.py` และ `build_docs_pdf.mjs` (เอกสาร)

16. ข้อจำกัดทางเทคนิคและงานต่อ

เรื่อง	รายละเอียด
ยังไม่ได้ทดสอบกับโมเดลจริงในรุ่นนี้	LIVE_FREE ถูกระงับที่ preflight: นโยบายยังไม่ทบทวน สถานะฟรีของบัญชียังไม่ยืนยัน และไม่มี key ทดลอง
Render แผนฟรี	หลับหลังไม่มีการใช้งาน การตื่นอาจทำให้ proxy ตอบ 502 ก่อนแอปพร้อม
`/ready` ขึ้นกับ PostgreSQL	deploy ใหม่จะรับงานเมื่อฐานข้อมูลตอบเท่านั้น
Storage ยังอยู่บน event loop บางจุด	route คืนเงินของเจ้าหน้าที่ worker ของ LINE (ปิดบน Render) และตัวจำลอง LINE
Windows	ขีดจำกัดหน่วยความจำของ process อ่านไฟล์ใช้ได้เฉพาะ POSIX
ค่าเริ่มต้นไม่ตรงกัน	`CLOUD_CALL_LIMIT` เป็น 200 ใน `config.py` แต่ 500 ใน `.env.example`
Rate limit	route ของ AI ใช้ token ของ rate limit 2 ครั้งต่อคำขอ
โค้ดเดิมที่ไม่อยู่ในเส้นทางจริง	`conversation_agent.run()`, `runtime_skills.bundle()` และ `image_validation.py`
งานระดับ P1	circuit breaker, นโยบายลองใหม่ และงานแบบ durable ยังไม่ได้ทำ

17. Checkpoint GL-03: bot/Admin Chat handoff

เส้นทาง handoff ผูก ticket กับ `chat_id` ของบทสนทนาที่ร้องขอจริง ไม่ใช้ active conversation ของเจ้าของเป็นค่าเดาเมื่อผู้ใช้สลับแชตแล้ว application fix หลักอยู่ใน `101e5e21968cc5ea8204f953ef60ceb9097bb677`; browser evidence hardening เดิมอยู่ใน `302c14e0b2f9abbdd88b4bb5dc57e31a8e282204`. รอบปัจจุบันบน commit `73d24b9cc4281b74829b63603786427d14cbe5c7` + dirty worktree เพิ่มการปิด transcript หลัง staff ปิด ticket และอนุญาตเริ่มแชตใหม่โดยไม่เปิด ticket เก่ากลับ

จุดตรวจ	ก่อนแก้	หลังแก้ใน `101e5e2`	หลักฐาน
Ticket binding	ticket ไม่มี `chat_id`; staff route อ่าน active chat ของ owner	ticket เก็บ `chat_id`, `mode_epoch`, `conversation_version` และ `assignment_epoch`	`routers/business.py`, focused pytest
Staff GET	อาจคืน transcript ของ active chat ที่ไม่ใช่เคส	lookup ตาม binding; missing binding/chat เป็น 404 และ archive ที่ตรงเท่านั้นถูกอ่านได้	`test_handoff_binds_exact_chat_and_stale_staff_mutations_fail_closed`
Staff state/reply	ไม่มี precondition สำหรับ request เก่า	รับ `expected_version` และ `assignment_epoch` แบบ optional; ค่าค้างคืน 409 ก่อนเขียนข้อมูล	focused pytest, UI-16
Takeover/return	เพิ่ม `version` แต่ไม่มีชื่อ epoch แยก	`mode_epoch` และ `version` เพิ่มพร้อมกัน ทำให้ finalization ของ bot ที่กำลังทำงานถูกทิ้ง	`test_staff_takeover_discards_inflight_model`
Client compatibility	payload เดิมมีเพียง state/message	optional fields ไม่ทำให้ client เดิมเสีย	focused tests และ UAT รอบล่าสุด
Closed chat	ปิด ticket แล้ว conversation กลับเป็น bot; chat/report actions เดิมยังทำงานได้ และ history อาจเปิดกลับเป็น bot	server ปฏิเสธ chat/upload/report-confirm โดยไม่ mutate; history คง closed; แชตใหม่ใช้ `chat_id` ใหม่	3 regression tests, full offline pytest; UI-16 screenshots ของ closed/history/new chat

การเปลี่ยนนี้ไม่เพิ่ม provider call ไม่เปิดเงินจริง และไม่เปลี่ยน package/payment simulator contract

17.1 ผล Playwright รอบใหม่

รอบ `final03` ใช้ local `OFFLINE/MOCKED_TEST_ONLY` บน Windows 11.0.26200 ด้วย Playwright 1.56.1, Chromium 141.0.7390.37, Node.js 24.19.0 และ Python 3.12.10; ใช้ข้อมูลสังเคราะห์, isolated temporary SQLite, ปิด provider networking และใช้ LLM/OCR doubles. Raw JSON ระบุ `37 PASS / 0 FAIL / 0 browser errors`, 63 screenshots และ exit code 0; checkpoint local UAT เป็น `PASS` โดยไม่ขยายผลไปยัง live provider หรือ production. Candidate commit คือ `73d24b9cc4281b74829b63603786427d14cbe5c7`.

Requirement	Test	ผลรอบนี้	Evidence
Customer/admin	UI-01–UI-13, UI-17, UI-23–UI-25, UI-28–UI-32	ทุก case ที่อยู่ใน raw PASS; ราคา live ไม่ได้เปลี่ยน	expected/actual รายcase
Bot↔admin handoff, close and resume	UI-16, UI-18, UI-34 และ GL-03 pytest	PASS ใน local fixture: privacy, reply, resume/requeue, close, history และ new-chat assertions ครบ	closed chat, raw JSON
History/privacy	UI-05, UI-14, UI-16, UI-19, UI-25, UI-31, UI-33 และ guest privacy tests	PASS สำหรับ local behavior ที่ทดสอบ; durable retention/restore NOT_RUN	guest screenshot, pytest XML, case results
Guest expiry/legacy purge	`PY-GUEST-EXPIRY-01`, `PY-GUEST-PURGE-01`	PASS ใน pytest; ไม่ใช่ Playwright	pytest XML

Requirement	Test	ผลรอบนี้	Evidence
Durable retention backup/restore	ไม่มี production backup/restore target	NOT_RUN	UAT scope and limitations
Package/payment แบบจำลอง	UI-02–UI-07, UI-15, UI-17, UI-26–UI-27 และ simulator expiry pytest	PASS แบบ simulator เท่านั้น; real settlement NOT_RUN	payment screenshot
Mobile/tablet	UI-21-tablet, UI-21-mobile, UI-31	PASS ตาม no-overflow/navigation assertions	mobile, tablet
Negative/permission	UI-02b, UI-04, UI-08, UI-16, UI-19–UI-20, UI-22, UI-33–UI-34	PASS ทุก case; raw มี 0 failed screenshots และ 0 browser errors	network error, permission gate

ขั้นตอน, expected/actual, status และ screenshot ต่อ case อยู่ใน UAT README และ raw JSON. Full offline pytest: **547 passed, 1 warning**, exit code 0; raw outputอยู่ที่ pytest XML.

PDF render รอบ `final5` และรายงานรอบ `final03` วิธีสร้างและตรวจภาพทุกหน้า, Thai text, pagination, theme/font fallback และ hash ระบุใน report render QA และ HTML deck QA.

17.2 Live provider smoke, ข้อจำกัดและ remaining risks

- `OFFLINE/MOCK` ยืนยันเฉพาะ UI/API/storage/permission; ไม่ยืนยัน live model/OCR quality, clinical accuracy, production uptime หรือ settlement
- Live smoke (แยกจาก OFFLINE/MOCK): Typhoon `LIVE-PROV-TYPHOON-01` (`2026-10-10T15:02:44Z`) และ iApp `LIVE-PROV-OPENTHAI-01` (`2026-10-10T15:18:57Z`) อย่างละ 1 request/HTTP 200; JSON ผ่านและแอปจัด

คำถามสังเคราะห์หนึ่งข้อว่า safe ยืนยัน connectivity/format เท่านั้น; iApp entitlement/debit ไม่ทราบ และไม่ได้เรียก OCR

- `NOT_RUN`/unknown: durable backup/restore, production retention, sequencing/race ภายใต้ production load, deploy/settlement, clinical/model/OCR quality, demand และ unit economics. Guest expiry/legacy purge เป็น PASS ใน offline pytest เท่านั้น ไม่ใช่ production-retention evidence
- UI-16 ผ่านใน final03 หลัง test รอ POST `/api/business/chats` และรอ composer กลับมาใช้งานก่อนอ่าน workspace; เป็นการแก้ test synchronization ไม่ใช่การลด assertion
- UI-28 ผ่านใน final03 บน isolated fixture; historical socket-hang ยังไม่มี root cause เชิง production จึงไม่ใช้ผลนี้อ้าง live reliability
- Review รอบนี้ให้ยึด exact dirty candidate ที่ระบุใน final03 raw JSON; ผล fresh review ต้องแยกจาก MAIN และระบุ reduced account independence หากใช้ Windows account/worktree เดียวกัน. การ push ทำหลัง hygiene และ review gate เท่านั้น

17.3 Checkpoint GL-05: manager-only simulated settlement

GL-05 เป็น authorization fence ของ endpoint simulator เดิม ไม่ใช่ระบบ bank settlement. `routers/business.py` ตรวจบทบาท manager ก่อน mutation ของ booking; `static/js/workspace.js` ใช้ capability เดียวกันเพื่อไม่แสดง action แก่ staff/clinical. `tests/browser/fixture_server.py` เพิ่มบัญชี staff สังเคราะห์สาขา BKK01 เพื่อไม่สับสนกับบัญชี manager ที่ใช้ในรอบก่อน.

Boundary	ก่อนแก้	หลังแก้และผลตรวจ	Status
API `/staff/bookings/{id}/settle`	non-manager mutation path ยังเป็น regression	`staff` และ `clinical` ได้ 403 ก่อนอ่าน/เขียน receipt; payment state, audit และ notification ไม่เปลี่ยน	PASS

Boundary	ก่อนแก้	หลังแก้และผลตรวจ	Status
Manager simulator	ต้องคง behavior เดิม	manager เห็นและใช้ action ที่ระบุ simulation; idempotent receipt เดิมยังทำงาน	PASS
Browser role/UI	fixture เดิมใช้ชื่อ staff แต่ role manager	UI-13 ยืนยัน session role จริง, staff ไม่เห็นปุ่ม, manager เห็นหลัง filter confirmed	PASS
Full offline browser	ต้องครอบคลุม customer/admin, handoff, privacy, package/payment, mobile และ negative cases	final03 raw: 37 PASS, 0 FAIL, 0 browser errors, exit code 0	PASS (local mocked UAT)

หลักฐานรอบ `final03` ใช้ candidate `73d24b9cc4281b74829b63603786427d14cbe5c7` + dirty worktree; source fingerprint, raw start-time diff SHA, status porcelain, environment, case results, screenshot hashes และผล focused/full pytest อยู่ใน UAT README/raw, pytest XML

`UI-16` และ `UI-28` ผ่านใน final03 local fixture; real settlement, retention/backup/restore, production finance RBAC, live model/OCR quality, production load และ deployment ยัง NOT_RUN. ไม่ได้ใช้ `.env`/live provider ในรอบ offline นี้ และไม่มีเงินเคลื่อนย้ายจริง. Fresh report render ตรวจแล้วตาม render QA

ภาคผนวก ก. เอกสารอ้างอิงใน repository

เรื่อง	ไฟล์
API และสถาปัตยกรรม	`docs/api.md`; `docs/architecture.md`
AI provider และ resilience/runbook	`docs/ai-providers.md`; `docs/operations/resilience.md`

เรื่อง	ไฟล์
Deploy, การทดสอบ และหลักฐาน	`docs/deploy/render.md`; `docs/testing.md`; `docs/evidence/current/`; `docs/evidence/history/`